A Closed-loop Basil Plant Digital Twin Framework

Rayan Contuliano Bravo[1], Gianluca Bontempi[1], and Pascal Tribel[1]

[1] Machine Learning Group - ULB, Brussels, Belgium

Abstract. Most agricultural digital twins stop at modeling and prediction without ever closing the loop with real actuation. We propose a working digital twin for a single basil plant, linking sensors and camera-based plant analysis to a quality-scored state representation, a modular data pipeline, a safety-gated controller, and a monitoring dashboard. Over a 30-day run ($\sim 150,000$ observations, including a dry-down experiment), the system produced health estimates and generated irrigation requests which were executed or rejected according to the configured actuation constraints. Despite the sensor and connectivity failures, the deployment demonstrated a functioning closed-loop. However, the health estimates remain biologically unvalidated.

Keywords: Digital Twin · Plant health monitoring · Data quality · Control

1 Introduction

Plants depend on many interacting environmental and biological variables, such as temperature, humidity, light, soil moisture, and visible growth [1–3]. Tracking these variables and taking action in real time is hard to do by hand, and it does not scale beyond a few plants. A digital twin addresses this problem by connecting observations to a live digital representation and, when control is permitted, to actions in the physical environment. Often, those actions will then shape the next observations [4].

Agricultural twins have been proposed at plant, greenhouse, and farm scales covering growth-stage reasoning, 3D plant modeling, simulation, model-based control, long-distance communication, large-area observation, and cloud processing [1, 5–8]. However, some systems focus primarily on modeling or monitoring [1, 8, 9], while others produce recommendations but leave the physical action to a person [5, 7, 8]. GreenhouseDT goes further by demonstrating model-based control, but its evaluation focuses on architectural extensibility and does not document an extended deployment with data qualification or explicit actuation limits [6]. More broadly, predictive, prescriptive, and autonomous actions across the complete life-cycle remain at an early stage, and many studies do not provide explicit architectures or technical designs [10].

In this study, we have implemented a digital twin with control actions in the case of a single basil plant. Automatic observation updates and physical actuation

provide the bidirectional integration described by Kritzinger et al.'s Digital Twin definition [4]. We study the choices and practical limits that arise when such a system is built and runs, and then present the measured data and actions that are taken on this basis, showing that this single-plant digital twin indeed implements a concrete closed-loop [1, 9, 10]. This evaluation concerns the implemented pipeline and does not establish the biological validity of the plant model.

2 Prototype

The physical twin is a single basil plant grown indoors in a wooden box. Basil suits this setup because it is a common indoor crop whose growth, development, and leaf color respond measurably to environmental conditions within weeks [11–13]. Sensors track soil moisture, temperature, humidity, and light, while cameras capture the plant's appearance. A water pump closes the loop as the actuator while a fan, heater, and LED light remain available for later use. A Raspberry Pi 4 runs the digital twin.

The digital twin mirrors this setup as a state $S(t) = \{E(t), O(t), Q(t)\}$: $E(t)$ holds the sensor readings, $O(t)$ the image-derived features (green-pixel ratio, leaf count, height), and $Q(t)$, a quality score $\in [0, 1]$, is the weighted average of passed range (0.4), rate-of-change (0.4) and stuck-value (0.2) checks[4]. Validated sensor and image events update $S(t)$, from which **Analytics** produces health estimates, forecasts and recommendations. Health estimation and irrigation recommendations use fixed thresholds, while the soil-moisture forecaster updates its RLS coefficients online. Atmospheric thresholds are informed by greenhouse and basil-production guidance [11, 13–15] while other health and irrigation thresholds are heuristics specific to this setup.

Fig. 1 summarizes the system, split into single-responsibility components. **Collector** publishes sensor readings and images. **Preprocessing** validates each reading and attaches a quality score. **Image processing** derives the green-pixel ratio by thresholding normalized ExG, counts top-view leaf detections with YOLO, and estimates side-view height from the ExG-segmented contour, scaled by an AprilTag. **Storage** handles time-series data and retains events for audit and future model improvement. **Analytics** then interprets this state: simple deterministic rules estimate health from hydration, atmospheric, visual, and data-quality features[5] , while an online Recursive Least Squares model forecasts soil moisture three hours ahead[6]. These analyses feed action recommendations. For instance, **Analytics** issues an 8-second pump request when health is *stressed* or *critical*, the soil moisture forecast is at most 55% and the lower of health and forecast confidence is at least 0.55. The **Controller** then acts as a safety gate, checking allowed commands (ON/OFF), a maximum duration of 15 seconds, a

[4]See the quality-score computation and the configured weights in the online repository.
[5]See the health-estimation diagram and group-specific diagrams that document the classification rules and thresholds in the project repository.
[6]See the prediction accuracy computation in the project repository.

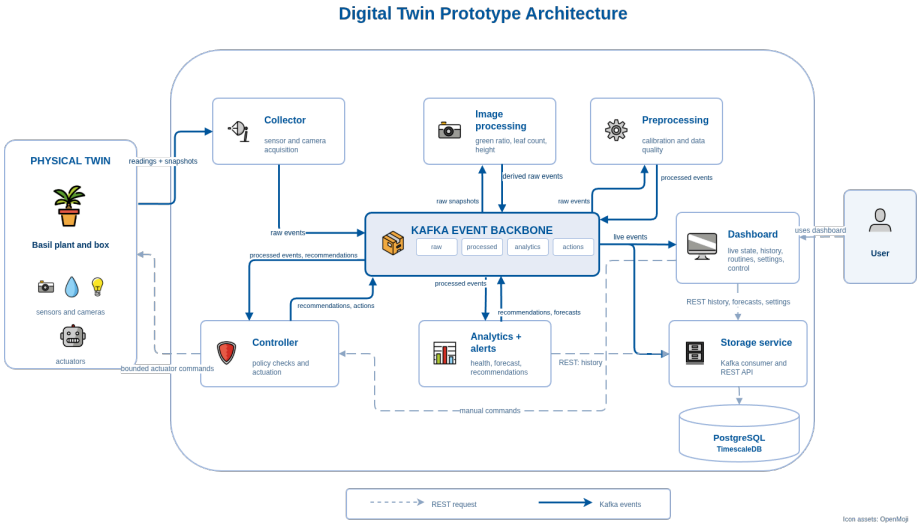


Fig. 1. Architecture of the prototype. Kafka carries live events between services, while REST supports targeted history, configuration, and command requests.

three-hour cooldown, and overlapping actions before executing anything. Finally, **Dashboard** exposes state, alerts, and action history, letting a human inspect or interact with the twin at any time.

3 Results

We ran the prototype from 15 April to 15 May 2026 on one basil plant, without a control plant, through development interruptions, power losses, and disconnected sensors and cameras. Even so, the system processed more than 150,000 observations, kept producing health estimates, and acted autonomously on the physical plant[7].

On 1,303 three-hour forecasts matched within five minutes, RLS had an MAE of 5.54 percentage points versus 4.65 for persistence[8]. The persistence baseline therefore performed better on this run.

To observe a noticeable change, we planned a period without watering, during which soil moisture fell toward 42%, the visible characteristics of the plant followed as the green-pixel ratio dropped from near 90% to below 50%, and the health estimator moved between *critical*, *stressed*, and *healthy*, reacting to changing conditions. Fig. 2 summarizes image-derived observations, health-state shares, and pump requests against the soil-moisture history. Four requests were rejected

[7]The source code and case study dataset are available in the project repository. The analysis script reproduces the reported counts and the data plotted in Fig. 2.

[8]Persistence predicts that soil moisture at $t + 3h$ equals its observed value at t.

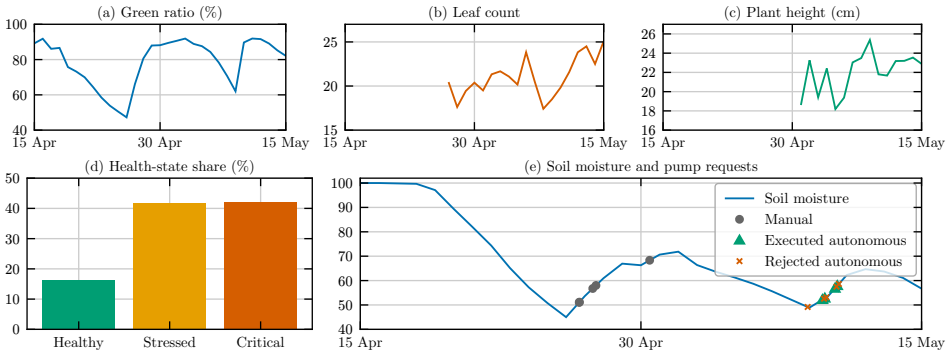


Fig. 2. Observations and actions during the 30-day run. Lines show daily medians. Markers show 5 manual pump activations and 9 autonomous requests (4 executed and 5 rejected)

during cooldown, and one because the requested 8 seconds exceeded a temporary 3-second maximum. During the period containing the executed autonomous actions, soil moisture changed from 48.9% to 64.7% and the green-pixel ratio from 62.1% to 91.9%[9].

4 Discussion

The deployment demonstrates an operational digital-twin pipeline. Each component has a single responsibility, supporting modularity and potential future scalability. Despite this design, several limitations remain. At the physical layer, hardware fragility is a vulnerability, with sensor and hardware disconnections, and lighting variations affecting the camera and CPU load on the Raspberry Pi, which can introduce latency and degrade data quality and performance. Moreover, current sensors capture only a fraction of the plant's physiological and structural traits. Modeling these dynamics and isolating specific causal effects remains difficult due to interacting environmental variables and delayed biological responses. With one plant, a 30-day run and no control plant, the experiment can't establish long-term plant-health benefits or an advantage over another plant-care strategy. As a result, the current analytics are not yet biologically validated. A 30-day run does not establish long-term operational reliability, so the prototype should be viewed as an experimental platform rather than a validated solution.

5 Conclusion

In this work, we presented the design and 30-day evaluation of a closed-loop digital twin for basil plant health monitoring. By deploying the prototype on a single basil plant, we demonstrated that the key strength of the prototype is its

[9]See the visual timeline, on 9-10 May, in the project repository.

modular architecture, combining multiple sensors, computer-vision features, time-series storage, and data qualification, which can produce health estimates and support autonomous irrigation under actuation constraints.

At the same time, the deployment highlighted several limitations that outline key areas for improvement. The health estimates remain biologically unvalidated, and the single-plant evaluation without a control plant does not establish improved plant health. The current analytics models also do not support simulations to compare alternative control strategies and inform decision-making. Future work will focus on hardening the hardware setup, improving the analytics through controlled agronomic experiments with appropriate control groups, and testing the platform across diverse plant species and longer life cycles.

Generative AI usage disclosure. ChatGPT was used to rephrase text for clarity and academic style and to help express existing concepts in clearer mathematical notation. GitHub Copilot was used for inline code completion during implementation. The authors reviewed and remain responsible for all content.

Bibliography

1. Mitsanis, C., Hurst, W., Tekinerdogan, B.: A 3D Functional Plant Modelling Framework for Agricultural Digital Twins. Computers and Electronics in Agriculture. 218, 108733 (2024). https://doi.org/10.1016/j.compag.2024.108733.
2. Melesse, T.Y.: Digital twin-based applications in crop monitoring. Heliyon. 11, e42137 (2025). https://doi.org/10.1016/j.heliyon.2025.e42137.
3. Kim, S., Heo, S.: An agricultural digital twin for mandarins demonstrates the potential for individualized agriculture. Nature Communications. 15, 1561 (2024). https://doi.org/10.1038/s41467-024-45725-x.
4. Kritzinger, W., Karner, M., Traar, G., Henjes, J., Sihn, W.: Digital Twin in Manufacturing: A Categorical Literature Review and Classification. IFAC-PapersOnLine. 51, 1016–1022 (2018). https://doi.org/10.1016/j.ifacol.2018.08.474.
5. Skobelev, P., Laryukhin, V., Simonova, E., Goryanin, O., Yalovenko, V., Yalovenko, O.: Multi-agent approach for developing a digital twin of wheat. In: 2020 IEEE International Conference on Smart Computing (SMARTCOMP). pp. 268–273 (2020). https://doi.org/10.1109/SMARTCOMP50058.2020.00062.
6. Kamburjan, E., Sieve, R., Prabhu Baramashetru, C., Amato, M., Barmina, G., Occhipinti, E., Broch Johnsen, E.: GreenhouseDT: An Exemplar for Digital Twins. In: Proceedings of the 19th International Symposium on Software Engineering for Adaptive and Self-Managing Systems. pp. 175–181. Association for Computing Machinery, Lisbon, AA, Portugal (2024). https://doi.org/10.1145/3643915.3644108.
7. Angin, P., Anisi, M.H., Göksel, F., Gürsoy, C., Büyükgülcü, A.: AgriLoRa: A Digital Twin Framework for Smart Agriculture. J. Wirel. Mob. Networks Ubiquitous Comput. Dependable Appl. 11, 77–96 (2020).
8. Madeira, R.N., Santos, P.A., Java, O., Priebe, T., Graça, E., Sárközi, E., Asprion, B., Gómez, R.P.-B.: Towards Digital Twins for Multi-Sensor Land and Plant Monitoring. Procedia Computer Science. 210, 45–52 (2022). https://doi.org/10.1016/j.procs.2022.10.118.
9. Verdouw, C., Tekinerdogan, B., Beulens, A., Wolfert, S.: Digital twins in smart farming. Agricultural Systems. 189, 103046 (2021). https://doi.org/10.1016/j.agsy.2020.103046.

10. Ariesen-Verschuur, N., Verdouw, C., Tekinerdogan, B.: Digital Twins in Greenhouse Horticulture: A Review. Computers and Electronics in Agriculture. 199, 107183 (2022). https://doi.org/10.1016/j.compag.2022.107183.
11. Walters, K.J., Currey, C.J.: Growth and Development of Basil Species in Response to Temperature. HortScience. 54, 1915–1920 (2019). https://doi.org/10.21273/HORTSCI12976-18.
12. Walters, K.J., Tarr, S., Lopez, R.G.: Modeling purple basil, sage, spearmint, and sweet basil responses to daily light integral and mean daily temperature. PLOS ONE. 18, (2023). https://doi.org/10.1371/journal.pone.0294905.
13. Currey, C.J.: Managing basil production throughout the year, https://www.producegrower.com/article/hydroponic-production-primer-managing-basil-production-throughout-the-year/, last accessed 2026/05/14.
14. Wollaeger, H., Runkle, E.: Why should greenhouse growers pay attention to vapor-pressure deficit and not relative humidity?, https://www.canr.msu.edu/news/why_should_greenhouse_growers_pay_attention_to_vapor_pressure_deficit_and_n, last accessed 2026/05/14.
15. Pundt, L.: Downy Mildew on Basil in the Greenhouse, https://extension.uconn.edu/publication/downy-mildew-on-basil-in-the-greenhouse/, last accessed 2026/05/14. https://doi.org/10.61899/ucext.v1.034.2024.